

Technische Referenz

TDS Toolkit **Einen Protokoll-Decoder schreiben**

Für Digitaloszilloskope der Serien TDS 500, TDS 600 und TDS 700

Dokument TDSTK-DEC-01
Copyright © 2026 Jared Cabot. MIT-Lizenz.

TDSTK-DEC-01

Copyright © 2026 Jared Cabot. Alle Rechte vorbehalten.

Dieses Dokument beschreibt die Decoder-Plug-in-Schnittstelle von TDS Toolkit. Die Informationen in dieser Veröffentlichung ersetzen alle zuvor veröffentlichten Angaben.

TDS Toolkit ist Host-Software; es ist kein Produkt von Tektronix. Tektronix und TDS werden nur verwendet, um die von der Software unterstützten Geräte zu benennen.

Diese Software wird unter der MIT-Lizenz veröffentlicht. DIE SOFTWARE WIRD OHNE MÄNGELGEWÄHR BEREITGESTELLT ("AS IS"), OHNE JEGLICHE AUSDRÜCKLICHE ODER STILLSCHWEIGENDE GEWÄHRLEISTUNG. Der vollständige Text liegt dem Programm als LICENSE bei.

Inhaltsverzeichnis

Zugehörige Handbücher	iv
1 Das Decoder-Modell	1-1
Was ein Decoder ist	1-1
Eine Datei, ein Protokoll	1-1
Der Vertrag	1-1
Auffinden	1-1
Wo die Dateien liegen	1-1
2 Das Signal	2-1
Was eine Dekodierung erhält	2-1
Abtastwerte und Zeitverhalten	2-1
Pegel, nicht Abtastwerte	2-1
Woher ein Signal stammt	2-2
3 Der Decoder-Vertrag	3-1
Klassenattribute	3-1
Die vier Attribute	3-1
Rollen und Quellen	3-1
Die Methoden	3-2
decode	3-2
Parameter	3-3
Der Typ Param	3-3
4 Rahmen und Darstellung	4-1
Der Frame	4-1
Ein dekodiertes Ereignis	4-1
Status	4-1
Das Ergebnis darstellen	4-2
Vier kleine Methoden	4-2
Farbe	4-2

5 Einen Decoder erstellen	5-1
Ein durchgearbeitetes Beispiel	5-1
Das Protokoll	5-1
Die Schritte	5-1
Der Code	5-1
Importe und Attribute	5-1
Die Bits zurückgewinnen	5-2
Die Wörter zusammenfassen	5-2
Darstellung	5-3
Die serielle Abkürzung	5-4
Die Bitabtastung nicht neu implementieren	5-4
6 Installieren und Testen	6-1
Installieren	6-1
Ablegen und öffnen	6-1
Testen	6-1
Mit einem synthetischen Signal antreiben	6-1
Jeden Decoder testen	6-2
Prüfliste	6-3
Bevor Sie sich darauf verlassen	6-3

Tabellenverzeichnis

Tabelle 1–1: Dateien des Decoder-Subsystems	1–2
Tabelle 2–1: Elemente eines Signal	2–2
Tabelle 3–1: Klassenattribute von Decoder	3–1
Tabelle 3–2: Methoden von Decoder	3–2
Tabelle 3–3: Param-Arten	3–3
Tabelle 4–1: Felder von Frame	4–1
Tabelle 4–2: Statusbits von Frame	4–2

Vorwort

Dieses Handbuch beschreibt, wie ein Protokoll-Decoder zu TDS Toolkit hinzugefügt wird. Ein Decoder ist eine einzige Python-Datei. Er liest einen aufgezeichneten Signalverlauf und gibt eine Liste dekodierter Ereignisse zurück; das Programm listet ihn auf, ordnet seine Eingänge zu, führt ihn aus und zeichnet das Ergebnis ohne weiteren Code.

Dieses Handbuch richtet sich an erfahrene Programmierer mit guten Python-Kenntnissen. Kenntnisse über den Rest des Programms sind nicht erforderlich: Die gesamte Schnittstelle besteht aus einer Basisklasse und drei kleinen Datentypen, allesamt im Modul `tds_decode.py`.

Lesen Sie dieses Handbuch zusammen mit zwei Dateien, die mit dem Programm ausgeliefert werden: `decoders/_template.py`, dem zu kopierenden Grundgerüst, und `decoders/README.txt`, der Kurzfassung des Folgenden.

Zugehörige Handbücher

Die Schnittstelle und die Dateien, die neben diesem Handbuch zu lesen sind:

- `decoders/_template.py` – das zu kopierende Grundgerüst.
- `decoders/README.txt` – die Kurzfassung des Folgenden.
- `tds_decode.py` – der Kern: die Basisklasse `Decoder` und die Typen `Signal`, `Frame` und `Param`.

Das Decoder-Modell

Was ein Decoder ist

Eine Datei, ein Protokoll

Ein Decoder ist eine einzige Python-Datei im Ordner decoders, die eine Unterklasse von Decoder definiert. Jede Datei ist ein Protokoll. Die Registerkarte Signalverläufe listet jeden gefundenen Decoder auf, sodass für ein neues Protokoll eine einzige Datei genügt und sich sonst nichts im Programm ändert. Eine Dekodierung läuft vollständig auf einem aufgezeichneten Signalverlauf ab, den die Registerkarte Signalverläufe bereits vorhält: einem vom Gerät gelesenen Kanal oder einer aus einer Datei geladenen gespeicherten Referenz. Nichts in einem Decoder greift auf das Gerät zu. Das ist beabsichtigt. Es funktioniert mit jedem TDS, einschließlich der Basismodelle, deren Firmware keine eigene Bus-Dekodierungsanwendung ausführen kann, weil die Dekodierung nach der Aufzeichnung im Host anhand der Abtastwerte erfolgt.

Der Vertrag

Die gesamte Schnittstelle ist die Basisklasse Decoder. Ein Decoder setzt vier Klassenattribute und implementiert eine Methode. Die anderen drei Methoden haben funktionierende Standardimplementierungen und werden nur überschrieben, um die eigenen Felder des Protokolls in der Tabelle und im Diagramm darzustellen. Die drei Datentypen, mit denen ein Decoder umgeht – Signal, Frame und Param –, werden in den folgenden Kapiteln beschrieben. Beim Start und immer, wenn die Registerkarte Signalverläufe geöffnet wird, durchsucht das Programm den Ordner decoders nach *.py-Dateien, importiert jede einzeln und instanziiert jede gefundene Decoder-Unterklasse. Eine Datei, deren Name mit einem Unterstrich beginnt, wird als Hilfsmodul behandelt und übersprungen; so bleibt _template.py eine Vorlage und erscheint nie als leeres Protokoll in der Liste.

Auffinden

Der Import ist defensiv. Eine Datei, die sich nicht importieren lässt, sei es wegen eines Syntaxfehlers oder einer fehlerhaften Referenz, wird mit einem Vermerk im Programmprotokoll übersprungen, statt die gesamte Liste mit sich zu reißen. Ein halbfertiger Decoder verdeckt niemals die funktionierenden. Wenn zwei Ordner einen Decoder mit demselben NAME anbieten, gewinnt der spätere Ordner, sodass eine lokal geschriebene Kopie die mitgelieferte überschreibt.

Wo die Dateien liegen

Die mit dem Programm ausgelieferten Decoder liegen in einem Ordner `decoders` innerhalb der Installation. Eine beschreibbare Kopie wird neben der Einstellungsdatei `tdstoolkit.json` vorgehalten und bei jedem Start aus dem mitgelieferten Satz ergänzt, genau wie die Maskenbibliothek. Ein in einem späteren Build hinzugefügter Decoder wird herüberkopiert, ohne einen lokal geschriebenen zu überschreiben; ein gelöschter mitgelieferter Decoder kehrt beim nächsten Start zurück, weil der mitgelieferte Satz vorhanden sein soll. Legen Sie eine neue Datei in den beschreibbaren Ordner, und sie erscheint beim nächsten Öffnen der Registerkarte in der Liste. Eine Neuerstellung ist nicht erforderlich.

Tabelle 1-1: Dateien des Decoder-Subsystems

Datei	Funktion
<code>tds_decode.py</code>	Der Kern: die Basisklasse <code>Decoder</code> , die Typen <code>Signal</code> , <code>Frame</code> und <code>Param</code> , die Statuskonstanten, die gemeinsame serielle Hilfsfunktion und das Auffinden. Ein Decoder importiert daraus und ändert es nicht.
<code>decoders/_template.py</code>	Das Grundgerüst. Kopieren Sie es, um einen neuen Decoder zu beginnen.
<code>decoders/README.txt</code>	Die Kurzanleitung und eine Übersicht der Beispiel-Decoder.
<code>decoders/.py</code>	Je ein Protokoll: <code>uart.py</code> , <code>i2c.py</code> , <code>spi.py</code> , <code>can.py</code> und die übrigen. Lesen Sie die Datei, die dem hinzuzufügenden Protokoll am nächsten kommt.

Das Signal

Was eine Dekodierung erhält

Abtastwerte und Zeitverhalten

Der Methode `decode()` wird ein Signal je Eingang übergeben, den das Protokoll benötigt. Ein Signal ist eine erfasste Aufzeichnung, reduziert auf das, was ein Decoder braucht: die ganzzahligen Abtastwerte und die Zeitinformation, die einen Abtastindex in eine Zeit umrechnet. Es enthält keine Spannungswerte und keine Anzeigeskalierung. Ein Decoder vergleicht einen Abtastwert mit einer Schwelle und muss die Spannung nie kennen, daher werden die rohen Digitalisierer-Zählwerte unskaliert durchgereicht und eine Aufzeichnung mit einer Viertelmillion Punkten kostet bei der Übergabe nichts.

Die Entscheidungsschwelle, das Hysteresis-Totband und die Signalspanne werden berechnet, bevor `decode()` aufgerufen wird. Ein Decoder berechnet keine eigene Schwelle. Die Schwelle ist der Mittelpunkt zwischen den Extremwerten der Aufzeichnung; die Hysteresis beträgt ein Achtel der Spanne, breit genug, um Flankenklinglein zu unterdrücken, aber schmal genug, dass auch eine langsame Flanke sie überschreitet. Diese Werte werden gelesen, nicht gesetzt.

Pegel, nicht Abtastwerte

Für die meisten Protokolle sind nicht die rohen Abtastwerte die nützliche Form der Aufzeichnung, sondern ein sauberer logischer Pegel je Abtastwert. Die Methode `digitize()` liefert genau das: eine 0 oder 1 für jeden Abtastwert, wobei die Hysteresis der Aufzeichnung angewendet wird, sodass Flankenklinglein nicht als mehrere Übergänge gelesen wird. Rufen Sie sie einmal auf und durchlaufen Sie das Ergebnis. Ein Protokoll, das die rohen Zählwerte benötigt, etwa eine bipolare Leitung mit zwei Schwellen, liest stattdessen `sig.data` direkt.

Tabelle 2–1: Elemente eines Signal

Element	Bedeutung
sig.data	Die ganzzahligen Abtastwerte, eine indizierbare Folge roher Digitalisierer-Zählwerte.
sig.n	Die Anzahl der Abtastwerte, dasselbe wie len(sig.data).
sig.dt	Sekunden pro Abtastwert.
sig.t0	Die Zeit des ersten Abtastwerts, in Sekunden relativ zum Trigger.
sig.first_index	Der Aufzeichnungs-Offset des ersten Abtastwerts. Addieren Sie ihn zu einem lokalen Abtastindex, um eine absolute Position in der Aufzeichnung zu erhalten.
sig.threshold	Der Entscheidungspegel, in Zählwerten. Vom Framework gesetzt.
sig.hysteresis	Das Totband um die Schwelle, in Zählwerten. Vom Framework gesetzt.
sig.span	Die Differenz zwischen dem höchsten und dem niedrigsten Abtastwert der Aufzeichnung. Vom Framework gesetzt; eine Spanne nahe null bedeutet eine flache, signalfreie Quelle.
sig.time_at(i)	Die Zeit in Sekunden am lokalen Abtastindex i, also $t_0 + i \times dt$.
sig.digitize()	Ein neues bytearray mit einem 0/1-Pegel je Abtastwert, mit Hysterese.
sig.auto_threshold()	Berechnet Schwelle, Hysterese und Spanne neu. Wird bereits vom Framework aufgerufen; nur bei einem von Hand erstellten Signal aufzurufen (siehe Abschnitt 6).

HINWEIS. Ein Signal enthält rohe Zählwerte, keine Volt. Vergleichen Sie mit sig.threshold, niemals mit einer Spannung. Ein Decoder, der bei einer vertikalen Skalierung korrekt liest, liest dann bei jeder anderen Skalierung korrekt, weil die Schwelle mit der Aufzeichnung mitwandert.

Woher ein Signal stammt

Das Signal, das decode() erhält, wird nicht vom Decoder, sondern vom Programm aus dem aufgezeichneten Signalverlauf erstellt, mit der Klassenmethode Signal.from_waveform(). Sie liest die ganzzahligen Pegel sowie das Abtastintervall und den Trigger-Offset aus der Präambel des Signalverlaufs, erstellt das Signal und ruft auto_threshold() darauf auf, bevor sie es an den Decoder übergibt. Ein Signal wird nur von Hand erstellt, wenn ein Test geschrieben wird, was Abschnitt 6 behandelt.

Der Decoder-Vertrag

Klassenattribute

Eine Decoder-Unterklasse setzt vier Klassenattribute. NAME und DESCRIPTION benennen sie in der Protokollliste. SOURCES deklariert die benötigten Signalverlauf-Eingänge. PARAMS deklariert die Einstellungen, die die Registerkarte anbietet. Alles Übrige wird geerbt.

Tabelle 3–1: Klassenattribute von Decoder

Attribut	Bedeutung
NAME	Der Protokollname. Wird in der Liste angezeigt und als Titel der Diagrammbeschriftung verwendet. Bleibt er leer oder "?", listet der Lader die Klasse nicht auf; so fehlt die Vorlage in der Liste.
DESCRIPTION	Eine Zeile für die zweite Spalte der Liste.
SOURCES	Eine Liste von Rollennamen, in fester Reihenfolge. Eine Rolle bedeutet ein Eindraht-Protokoll; zwei oder mehr erhalten jeweils eine zugeordnete aufgezeichnete Quelle. Ein abschließendes "?" kennzeichnet eine optionale Rolle, die ohne Zuordnung bleiben darf.
PARAMS	Eine Liste von Param-Objekten, je ein Bedienelement. Darf leer sein.
COLOUR	Die Winkelfarbe für einen sauberen Rahmen, als "#rrggbb". Ein Decoder besitzt seine Farben selbst; das Programm hat keine Farbeinstellung für Decoder. Setzen Sie sie für eine einheitliche Farbe des gesamten Protokolls.
FAULT_COLOUR	Die Winkelfarbe für einen fehlerhaften Rahmen. Standardmäßig rot, sodass ein Fehler bei der Farbe jedes Protokolls hervorsticht.

Rollen und Quellen

SOURCES ist zugleich die Anzahl der Signalverläufe, die ein Protokoll benötigt, und deren Namen. Ein Eindraht-Protokoll deklariert eine Rolle, üblicherweise "Line". Ein Zweidraht-Protokoll wie I2C deklariert ["SDA", "SCL"]; die Registerkarte zeigt einen Quellenwähler je Rolle, und decode() erhält für jede ein Signal. Ein abschließendes Fragezeichen, wie bei SPI mit ["CLK", "MOSI", "MISO?", "CS?"], kennzeichnet eine Rolle, die der Bediener ohne Zuordnung lassen darf; ihr Signal fehlt dann in dem Dictionary, das decode() erhält, und decode() muss dies prüfen.

Die Methoden

decode `decode(self, signals, params)` ist die einzige erforderliche Methode. `signals` ist ein Dictionary, das jeden Rollennamen, ohne ein abschließendes "?", auf sein Signal abbildet; eine optionale Rolle, die nicht zugeordnet wurde, fehlt. `params` ist ein Dictionary, das jeden Param-Schlüssel auf den vom Bediener gewählten Wert abbildet. Die Methode gibt ein Zwei-Tupel zurück: eine Liste von Frame-Objekten und einen einzeiligen Vermerk als Zeichenkette.

Der Vermerk wird über der Ergebnistabelle angezeigt. Verwenden Sie ihn, um zu melden, was die Dekodierung ermittelt hat, etwa die erkannte Bitrate, die Abtastwerte pro Bit und die Anzahl der gefundenen Rahmen, oder, wenn die Aufzeichnung nicht gelesen werden kann, um den Grund anzugeben. Geben Sie im Fehlerfall eine leere Rahmenliste und einen Vermerk zurück, der das Problem in einem Satz erklärt.

Tabelle 3–2: Methoden von Decoder

Methode	Rückgabe
<code>decode(signals, params)</code>	<code>(frames, note)</code> . Die Dekodierung selbst. Erforderlich.
<code>columns()</code>	Die Tabellenüberschriften nach der Spalte Zeit, als Liste von (Überschrift, Breite_in_Pixeln)-Paaren. Überschreiben, um die Felder des Protokolls zu benennen.
<code>row(frame, ascii)</code>	Die Tabellenzellen für einen Rahmen, passend zu <code>columns()</code> nach der Zeit. Überschreiben, um die Felder des Protokolls zu formatieren.
<code>label(frame, ascii)</code>	Die kürzeste nützliche Zeichenkette für den Winkel im Diagramm. Überschreiben, um sie knapp zu halten.
<code>colour(frame)</code>	Die Winkelfarbe für einen Rahmen. Standardmäßig COLOUR für einen sauberen und FAULT_COLOUR für einen fehlerhaften Rahmen. Überschreiben, um je Winkel einzufärben, anhand von <code>frame.kind</code> .

Das an `row()` und `label()` übergebene Argument `ascii` ist der Zustand des Hex/ASCII-Umschalters der Registerkarte. Berücksichtigen Sie es, wo es sinnvoll ist – ein Byte als Zeichen oder als zwei Hexadezimalziffern dargestellt –, und ignorieren Sie es, wo nicht.

Parameter

Jedes Param in der Liste PARAMS wird zu einem Bedienelement auf der Registerkarte, sodass ein Protokoll seine eigenen Optionen ohne jeglichen Code für die Benutzeroberfläche hinzufügt. Der Konstruktor lautet `Param(key, label, kind, choices, default, lo, hi, unit, note)`. `key` ist der Dictionary-Schlüssel, den `decode()` liest; `label` ist die Beschriftung, die der Bediener sieht; `kind` ist eine von drei Zeichenketten.

Tabelle 3–3: Param-Arten

kind	Bedienelement und Felder
"choice"	Eine Auswahlliste. <code>choices</code> ist eine Liste von (value, label)-Paaren; <code>value</code> ist der Wert, der in <code>params</code> ankommt, <code>label</code> der angezeigte Text. <code>default</code> ist einer der Werte.
"int"	Ein Ganzzahlfeld. <code>lo</code> und <code>hi</code> begrenzen es; <code>unit</code> ist ein Beschriftungssuffix wie "bits". Ein <code>default</code> von 0 wird üblicherweise als "auto" verstanden, was in <code>note</code> vermerkt werden sollte.
"bool"	Ein Kontrollkästchen. <code>default</code> ist <code>True</code> oder <code>False</code> .

Der Typ Param

Wird `default` weggelassen, gilt bei einem `choice` die erste Auswahl, bei einem `bool` `False` und bei einem `int` 0. Die optionale `note` wird als Hinweis neben dem Bedienelement angezeigt.

Rahmen und Darstellung

Der Frame

Ein dekodiertes Ereignis

Ein Frame ist ein dekodiertes Ereignis: ein UART-Zeichen, ein I2C-Byte, ein SPI-Wort, ein Buszustand. Der Typ ist flach und wird von jedem Protokoll gemeinsam genutzt. Er enthält ein primäres Datum, ein optionales sekundäres, eine Länge, einen Status und eine Art (kind). Was die Felder bedeuten, ist Sache des Decoders, da nur der Decoder, der einen Rahmen erzeugt hat, ihn darstellt, aber ihre Form liegt fest.

Erzeugen Sie einen Frame mit Schlüsselwortargumenten; jedes Feld hat eine Vorgabe. Die beiden Indexfelder sind am wichtigsten und am fehleranfälligsten. `index` und `end_index` sind absolute Abtastpositionen in der Aufzeichnung: Das Diagramm zeichnet das Winkelkästchen von einer zur anderen. Bilden Sie sie als `sig.first_index` plus den lokalen Abtastindex, genau so, wie die Zeit aus `sig.time_at()` des lokalen Index kommt.

Tabelle 4–1: Felder von Frame

Feld	Bedeutung
<code>time</code>	Die Startzeit des Ereignisses in Sekunden, aus <code>sig.time_at(local_index)</code> .
<code>index</code>	Der absolute Startabtastwert: <code>sig.first_index</code> + lokaler Start.
<code>end_index</code>	Der absolute Endabtastwert: <code>sig.first_index</code> + lokales Ende. Das Kästchen reicht von <code>index</code> bis <code>end_index</code> .
<code>value</code>	Das primäre Datum: das Byte, das Wort, der gemessene Wert.
<code>tag</code>	Ein optionales sekundäres Datum: eine I2C-Adresse, ein SPI-MISO-Wort, eine gemessene Frequenz. 0, wenn ungenutzt.
<code>length</code>	Die Länge des Datums, in der Einheit, in der das Protokoll zählt. Standardwert 1.
<code>status</code>	Ein bitweises ODER der Statuskonstanten; OK (null), wenn der Rahmen sauber ist.
<code>kind</code>	Eine freie Zeichenkette, etwa "byte", "addr", "data", "start" oder "stop", anhand derer <code>row()</code> und <code>label()</code> unterscheiden, um jeden Teil eines Rahmens für sich darzustellen.

Status status ist eine Bitmaske. Ein sauberer Rahmen ist OK, also null. Ein Rahmen mit einem Problem verknüpft eine oder mehrere der Konstanten in Tabelle 4-2 mit ODER. Das Framework zeichnet jeden Rahmen, dessen status ungleich null ist, als Fehler – einen roten Winkel und ein vorangestelltes Ausrufezeichen in der Beschriftung –, ohne dass der Decoder mehr tut, als das Bit zu setzen.

Tabelle 4-2: Statusbits von Frame

Konstante	Wert	Bedeutung
OK	0	Sauber; kein Fehler.
FRAMING_ERROR	1	Der Rahmen wurde nicht wie von seinem Format verlangt abgeschlossen, etwa ein UART-Stoppbit, das kein Mark war.
PARITY_ERROR	2	Ein Paritätsbit stimmte nicht mit den Daten überein.
TRUNCATED	4	Die Aufzeichnung endete mitten im Rahmen.
ACK_ERROR	8	Ein Byte, das eine Bestätigung erwartete, wurde nicht bestätigt.

Importieren Sie die verwendeten Konstanten aus `tds_decode` zusammen mit `Decoder`, `Frame` und `Param`. Definieren Sie keine weiteren Statuswerte; die Darstellung versteht nur die fünf obigen.

Das Ergebnis darstellen

Vier kleine Methoden

Die Basisklasse stellt einen Rahmen als einfaches Byte dar: Hex, ASCII und ein Statuswort. Überschreiben Sie `columns()`, `row()` und `label()`, um stattdessen die Begriffe des Protokolls zu verwenden. `columns()` liefert die Überschriften nach der Spalte Zeit, jede mit einer Pixelbreite. `row()` liefert eine Zelle je Überschrift, in derselben Reihenfolge. `label()` liefert die kürzeste nützliche Zeichenkette für den Winkel, der auf dem Signalverlauf selbst gezeichnet wird.

Ein Protokoll, dessen Rahmen unterschiedliche Arten tragen, unterscheidet in diesen Methoden anhand von `frame.kind`, sodass ein START, ein Adressbyte und ein Datenbyte in der Tabelle und im Diagramm jeweils als das erscheinen, was sie sind. Halten Sie `label()` knapp: Es wird in einem kleinen Kästchen über der Kurve gezeichnet, ein bis drei Zeichen sind daher die richtige Länge.

Farbe

Die vierte Methode, `colour(frame)`, legt die Winkelfarbe fest, und der Decoder besitzt sie vollständig: Das Programm hat keine Farbeinstellung für Decoder. Die Standardimplementierung zeichnet einen sauberen Rahmen in der Klassenfarbe `COLOUR` und einen fehlerhaften in `FAULT_COLOUR`, sodass das Setzen von `COLOUR` dem gesamten Protokoll eine Farbe gibt. Überschreiben Sie `colour()` und unterscheiden Sie anhand von `frame.kind`, um jedem Winkel eine eigene Farbe zu geben, sodass Adresse, Daten und Rahmung einer Leitung jeweils in einer eigenen Farbe erscheinen. Geben Sie ein `"#rrggbb"` zurück; greifen Sie für die Arten, die der Decoder nicht selbst einfärbt, auf die Basisklasse zurück.

Einen Decoder erstellen

Ein durchgearbeitetes Beispiel

Das Protokoll

Dieses Kapitel baut einen vollständigen Decoder aus der Vorlage auf. Das Beispielprotokoll ist eine einzelne selbsttaktende Leitung, die ein Bit pro Impuls überträgt: Der Pegel steigt, bleibt für einen Bruchteil der Bitperiode hoch und fällt; ein kurzes High ist eine 0 und ein langes High eine 1, entschieden danach, ob die High-Zeit mehr als die Hälfte der Periode beträgt. Bits werden zu Wörtern einer gewählten Größe und Bitreihenfolge zusammengefasst. Dies ist die Familie der WS2812-Pixelleitung und der NEC-Infrarot-Fernbedienung, und es gehört nicht zu den eingebauten Protokollen, sodass es ein echter neuer Decoder ist und nicht die Kopie eines vorhandenen.

Die Schritte

1. *Die Vorlage kopieren:* Kopieren Sie `decoders/_template.py` nach `decoders/pulsebits.py`.
2. *Das Protokoll benennen:* Benennen Sie die Klasse um und setzen Sie `NAME`, `DESCRIPTION` und `SOURCES`.
3. *Die Einstellungen deklarieren:* Füllen Sie `PARAMS` mit den Bedienelementen für Wortgröße und Bitreihenfolge.
4. *Die Dekodierung schreiben:* Gewinnen Sie die Bits aus dem Pegelverlauf zurück und fassen Sie sie dann zu Wörtern zusammen.
5. *Die Felder darstellen:* Überschreiben Sie `columns()`, `row()` und `label()`, um das Wort in Hex und Binär anzuzeigen.
6. *Testen:* Speisen Sie den Decoder mit einem synthetischen Signal und prüfen Sie das Ergebnis, wie Abschnitt 6 zeigt.

Der Code

Importe und Attribute

Der Dateikopf importiert nur aus `tds_decode`, benennt die Klasse und deklariert die beiden Eingaben, die die Registerkarte anbieten wird: eine Wortgröße und eine Bitreihenfolge. `SOURCES` enthält eine Rolle, sodass die Registerkarte einen einzigen Quellenwähler zeigt.

```

from tds_decode import Decoder, Frame, Param
from tds_decode import OK, TRUNCATED

class PulseBitsDecoder(Decoder):

    NAME = "Pulse-width bits"
    DESCRIPTION = "Self-clocked pulse-width line, one wire"

    SOURCES = ["Line"]
    PARAMS = [
        Param("bits", "Word size", "int", default=8,
              lo=1, hi=32, unit="bits"),
        Param("order", "Bit order", "choice",
              choices=[("msb", "MSB first"),
                       ("lsb", "LSB first")],
              default="msb"),
    ]

```

Die Bits zurückgewinnen

decode() nimmt das eine Signal, fängt eine flache Quelle ab und reduziert die Aufzeichnung mit digitize() auf einen 0/1-Pegel je Abtastwert. Dann durchläuft es den Pegelverlauf. Jedes Bit beginnt an einer steigenden Flanke; der Lauf hoher Abtastwerte ist der Impuls, der darauf folgende Lauf niedriger Abtastwerte reicht bis zur nächsten steigenden Flanke, und beide zusammen bilden die Bitperiode. Das Verhältnis von High-Zeit zu Periode entscheidet über das Bit.

```

def decode(self, signals, params):
    sig = signals["Line"]
    if sig.span < 8:
        return [], "no signal on this source"
    level = sig.digitize()
    n = len(level)
    word_bits = max(1, int(params.get("bits") or 8))
    msb = params.get("order", "msb") == "msb"

    bits = [] # (value, start_index, end_index)
    i = 1
    while i < n:
        if level[i] and not level[i - 1]: # a rising edge
            start = i
            while i < n and level[i]: # the high pulse
                i += 1
            high = i - start
            while i < n and not level[i]: # low, to next rise
                i += 1
            period = i - start
            if period > 0:
                bit = 1 if high * 2 > period else 0
                bits.append((bit, start, i))
        else:
            i += 1

```

Die Wörter zusammenfassen

Die Bits werden dann in Gruppen von `word_bits` genommen, je nach Parameter mit dem höchst- oder niedrigstwertigen Bit zuerst. Eine Gruppe, die zu kurz ausfällt, weil die Aufzeichnung endete, wird dennoch ausgegeben, mit ihrem Status als TRUNCATED markiert, sodass die Darstellung sie als Fehler zeichnet. Jeder Frame nimmt seine Zeit vom ersten Bit und reicht vom Beginn des ersten Bits bis zum Ende des letzten Bits. Der Vermerk meldet die Gesamtzahlen.

```
frames = []
for base in range(0, len(bits), word_bits):
    chunk = bits[base:base + word_bits]
    value = 0
    for pos, (bit, _s, _e) in enumerate(chunk):
        shift = (word_bits - 1 - pos) if msb else pos
        value |= bit << shift
    start_local = chunk[0][1]
    end_local = chunk[-1][2]
    ok = len(chunk) == word_bits
    frames.append(Frame(
        time=sig.time_at(start_local),
        index=sig.first_index + start_local,
        end_index=sig.first_index + end_local,
        value=value, length=len(chunk),
        status=OK if ok else TRUNCATED,
        kind="data"))
return frames, "%d bit(s), %d word(s)" % (
    len(bits), len(frames))
```

Darstellung

Die drei Darstellungsmethoden zeigen das Wort als Hex und als Binärzahl seiner eigenen Bitbreite und kennzeichnen ein abgeschnittenes Wort. `label()` beschränkt sich auf zwei Hexadezimalziffern mit einem vorangestellten Ausrufezeichen bei einem Fehler, was der Konvention entspricht, der die eingebauten Decoder folgen.

```
def columns(self):
    return [("Time", 92), ("Hex", 60),
            ("Binary", 80), ("Status", 70)]

def row(self, frame, ascii=True):
    return ["0x%02X" % frame.value,
            format(frame.value, "0%db" % frame.length),
            "OK" if frame.status == OK else "CUT"]

def label(self, frame, ascii=True):
    text = "%02X" % (frame.value & 0xFF)
    return ("!" + text) if frame.status else text
```

Diese Datei ist ein vollständiger, auflistbarer Decoder. In den Ordner `decoders` gelegt, erscheint sie in der Registerkarte als "Pulse-width bits" mit einem Wortgrößenfeld und einem Bitreihenfolge-Dropdown und dekodiert jede Quelle, die ihrem einen Eingang zugeordnet ist.

Die serielle Abkürzung

Die Bitabtastung nicht neu implementieren

Das durchgearbeitete Beispiel tastet Bits selbst ab, weil sein Protokoll selbsttaktend und mit nichts Eingebautem vergleichbar ist. Für alles aus der Familie der asynchronen seriellen Protokolle, etwa RS-232, RS-422/485, MODBUS, MIDI, LIN, XBee und DMX512, implementieren Sie die Bitabtastung nicht neu. Rufen Sie die gemeinsame, am Korpus getestete Hilfsfunktion `serial_bytes()` auf, die Auto-Baud, Abtastung in der Bitmitte sowie Paritäts- und Stoppbitprüfungen auf die eine bewährte Weise durchführt, und bauen Sie die Rahmung auf dem von ihr zurückgegebenen Bytestrom auf.

```
from tds_decode import Decoder, Param
from tds_decode import serial_bytes, STANDARD_BAUDS

class MyUartDecoder(Decoder):

    NAME = "My UART"
    DESCRIPTION = "Async serial, one wire"
    SOURCES = ["Line"]
    PARAMS = [
        Param("baud", "Bit rate", "choice",
              choices=[(0, "Auto")]
              + [(b, str(b)) for b in STANDARD_BAUDS],
              default=0),
        Param("framing", "Framing", "choice",
              choices=[(f, f) for f in
                       ("8N1", "8E1", "8O1", "8N2")],
              default="8N1"),
    ]

    def decode(self, signals, params):
        sig = signals["Line"]
        frames, err, baud, spb = serial_bytes(
            sig, int(params.get("baud") or 0),
            params.get("framing", "8N1"), inverted=False)
        if err:
            return [], err
        return frames, "%d baud, %.1f smp/bit, %d frames" % (
            baud, spb, len(frames))
```

`serial_bytes(sig, baud, framing, inverted)` gibt vier Werte zurück: die Byte-Rahmen, eine Fehlerzeichenkette, die bei Erfolg leer ist, die erkannte Baudrate und die Abtastwerte pro Bit. Mit `baud = 0` wird die Rate automatisch erkannt. Mit `inverted = True` wird eine Polarität mit Ruhepegel Low und aktivem High gelesen, wie bei echtem RS-232. Ein höherstufiges Protokoll wie MODBUS oder LIN ruft `serial_bytes` für die rohen Bytes auf und gruppiert und prüft sie dann zu seinen eigenen Rahmen; lesen Sie `modbus.py` und `lin.py` für dieses Muster.

Installieren und Testen

Installieren

Ablegen und öffnen

Kopieren Sie die fertige Datei in den beschreibbaren Ordner decoders neben tdstoolkit.json und öffnen Sie die Registerkarte Signalverläufe (oder öffnen Sie sie erneut). Der Decoder erscheint in der Protokollliste. Es gibt keinen Registrierungsschritt und keine Neuerstellung. Ordnen Sie jeder seiner Rollen eine aufgezeichnete Quelle zu, setzen Sie seine Parameter und führen Sie ihn aus.

HINWEIS. Wenn der Decoder nicht erscheint, wurde er nicht importiert. Prüfen Sie das Programmprotokoll auf eine Zeile, die die Datei und die Ausnahme nennt. Der Lader überspringt eine Datei, die sich nicht importieren lässt, statt die gesamte Liste abubrechen; ein fehlerhafter Decoder fällt daher still aus.

Testen

Mit einem synthetischen Signal antreiben

Ein Decoder ist eine reine Funktion seiner Signale und Parameter, daher lässt er sich ohne Gerät testen, indem man ein Signal aus einem konstruierten Abtastwert-Array aufbaut, decode() aufruft und das Ergebnis prüft. Bauen Sie die Abtastwerte so, dass ihre Bedeutung bekannt ist, verpacken Sie sie in einem Signal, rufen Sie auto_threshold() auf – den einen Schritt, den das Framework normalerweise übernimmt – und prüfen Sie die Rahmen.

```

from tds_decode import Signal
from pulsebits import PulseBitsDecoder

def _bit(value, period=40):
    """One pulse-width bit: high for part of the period."""
    high = int(period * (0.66 if value else 0.33))
    return [30000] * high + [-30000] * (period - high)

def demo():
    stream = [-30000] * 20          # idle low lead-in
    for bit in (1, 0, 1, 1, 0, 0, 1, 0): # 0xB2, MSB first
        stream += _bit(bit)
    stream += [-30000] * 20        # idle low tail
    sig = Signal(stream, 1e-6)     # 1 us per sample
    sig.auto_threshold()           # framework does this
    frames, note = PulseBitsDecoder().decode(
        {"Line": sig}, {"bits": 8, "order": "msb"})
    assert len(frames) == 1, note
    assert frames[0].value == 0xB2, "%02X" % frames[0].value
    print("ok:", note)

if __name__ == "__main__":
    demo()

```

Jeden Decoder testen

Jeder eingebaute Decoder wird auf diese Weise geprüft. Der RS-232-Decoder wird gegen einen Korpus echter Aufzeichnungen ausgeführt; jeder andere Decoder wird gegen einen synthetischen Vektor in `tdstest.py` geprüft, jeweils mit einem passenden Mutationsbiss in `bitecheck.py`, der den Decoder absichtlich verfälscht und bestätigt, dass der Test es bemerkt. Legen Sie für jedes Protokoll, das Sie hinzufügen, einen Testfall an.



VORSICHT. Eine ungetestete Dekodierung weicht mit der Zeit unbemerkt ab. Fügen Sie für jeden hinzugefügten Decoder einen synthetischen Vektor und einen Mutationsbiss hinzu, der beweist, dass der Vektor tatsächlich etwas prüft, bevor Sie sich auf das Ergebnis verlassen.

Prüfliste

Bevor Sie sich darauf verlassen

- Die Datei ist in Kleinbuchstaben ohne Leerzeichen benannt und beginnt nicht mit einem Unterstrich.
- Die Klasse setzt NAME, DESCRIPTION, SOURCES und PARAMS, und NAME ist nicht leer.
- decode() gibt auf jedem Pfad ein (frames, note)-Tupel zurück, auch auf den Fehlerpfaden.
- Frame-index und -end_index sind absolut: sig.first_index plus der lokale Abtastindex.
- Eine nicht lesbare Aufzeichnung ergibt eine leere Liste und einen Vermerk mit der Begründung, niemals eine Ausnahme.
- columns(), row() und label() stimmen in Anzahl und Reihenfolge der Felder überein.
- Ein synthetischer Test treibt den Decoder an und prüft die dekodierten Werte.